

REPORT

ARTIFICIAL INTELLIGENCE (CSA1704)

Assessment Tool 3 – Integrated Experiments Report

Student Name: Madhu Bavana.B

Register Number:192524103

Course Code: CSA1704

Course Name: Artificial Intelligence

Assessment Title: Integrated Experiments Report

TABLE OF CONTENTS

NO	CONTENT	PAGE NO
1	Introduction	2
2	Decision Tree Classification	2
3	Q-Learning – Grid World	4
4	Conclusion	6
5	References	7

INTRODUCTION

1. Decision Tree Classification

Decision Tree Classification is a supervised machine learning technique used to classify data into different categories based on input features. It represents the decision-making process in a tree-like structure consisting of a root node, internal nodes, branches, and leaf nodes. The algorithm selects the most suitable feature for splitting the data using criteria such as Gini Index or Information Gain. In this experiment, the Iris dataset is used to train a Decision Tree classifier. The model is evaluated using accuracy, confusion matrix, precision, recall, and F1-score to determine its classification performance.

2. Q-Learning – 4×4 Grid World

Q-Learning is a model-free reinforcement learning algorithm in which an agent learns the best actions through interaction with an environment. The agent receives rewards or penalties based on its actions and gradually builds a Q-table containing the expected future reward for each state-action pair. In this experiment, a 4×4 Grid World is used, where the agent must navigate from a starting state to a goal state while avoiding an obstacle. The agent learns through repeated trials using the ϵ -greedy exploration strategy, and its learning progress is analyzed using Q-values and cumulative rewards over 100 episodes.

INTEGRATED EXPERIMENTS REPORT

Experiment 1: Decision Tree Classification

1. Aim

To implement and evaluate a Decision Tree Classification model using a suitable dataset and analyze its classification performance.

2. Objective

- To load and preprocess the dataset.
- To handle missing values and encode categorical data when required.
- To divide the dataset into training and testing sets.
- To construct a Decision Tree using Gini Index or Information Gain.
- To evaluate the classifier using a confusion matrix and classification report.

- To identify misclassified instances.

3. Theory

A Decision Tree is a supervised machine learning algorithm used for classification and prediction. It represents decisions in the form of a tree consisting of a root node, internal decision nodes, branches, and leaf nodes.

The algorithm selects the best feature for splitting the dataset using measures such as **Gini Index** or **Information Gain**.

Gini Index:

$$\text{Gini} = 1 - \sum(p_i^2)$$

where p_i represents the probability of a sample belonging to class i .

Information Gain:

$$\text{Information Gain} = \text{Entropy}(\text{parent}) - \text{Weighted Entropy}(\text{children})$$

A good split produces purer child nodes and improves classification accuracy.

4. Methodology

1. Load the selected dataset using Python.
2. Inspect the first five rows and data types.
3. Check and handle missing values.
4. Encode categorical attributes if necessary.
5. Separate input features and target labels.
6. Split the dataset into training and testing data.
7. Create the Decision Tree classifier.
8. Train the model using the training data.
9. Display the Decision Tree structure.
10. Predict the classes of the test data.
11. Generate the confusion matrix and classification report.
12. Analyze correctly and incorrectly classified instances.

5. Result

The Decision Tree classifier successfully learned the relationship between the input features and target classes. The confusion matrix and classification report

were used to measure the model's performance using accuracy, precision, recall, and F1-score. Misclassified test instances were also identified.

6. Conclusion

The Decision Tree Classification experiment demonstrates how a supervised learning algorithm can classify data using hierarchical decision rules. The model can provide an interpretable tree structure and useful classification performance. The evaluation results show the effectiveness of the selected features and splitting criterion for classification.

Experiment 2: Q-Learning – Grid World

1. Aim

To implement Q-Learning for a 4×4 Grid World environment and observe the agent's learned optimal policy.

2. Objective

- To define the Grid World states and actions.
- To specify the goal, obstacle, and reward structure.
- To initialize the Q-table.
- To train the agent for at least 100 episodes.
- To record Q-values at Episodes 1, 50, and 100.
- To determine the optimal action for each state.
- To plot cumulative rewards and analyze convergence.

3. Theory

Q-Learning is a model-free reinforcement learning algorithm. An agent learns the best action to take in a particular state by repeatedly interacting with the environment.

The Q-value represents the expected future reward for taking an action in a particular state.

The Q-Learning update equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where:

- s = current state
- a = selected action
- r = reward received
- s' = next state
- α = learning rate
- γ = discount factor

The agent gradually improves its decisions based on rewards received from the environment.

4. Environment

A 4×4 Grid World contains 16 possible states.

Actions:

- Up
- Down
- Left
- Right

Reward structure:

- Goal state: **+10**
- Each normal step: **-1**
- Obstacle: **-5**

The agent starts from a specified initial state and attempts to reach the goal while avoiding obstacles and minimizing unnecessary movements.

5. Methodology

1. Create a 4×4 Grid World.
2. Define all possible states and actions.
3. Specify the goal and obstacle states.
4. Assign rewards for different actions.
5. Initialize the Q-table with zeros.

6. Select suitable values for learning rate, discount factor, and exploration rate.
7. Start the agent from the initial state.
8. Select actions using an ϵ -greedy strategy.
9. Update the Q-value after every action.
10. Repeat the process for at least 100 episodes.
11. Record Q-values at Episodes 1, 50, and 100.
12. Extract the optimal action from the final Q-table.
13. Display the learned policy.
14. Plot cumulative reward for each episode.
15. Analyze the learning and convergence behavior.

6. Result

After repeated training episodes, the Q-table was updated based on the rewards obtained by the agent. The Q-values increased for actions that led toward the goal and decreased or remained low for undesirable actions. The final policy selected the action with the highest Q-value for each state.

The cumulative reward generally improved as the number of training episodes increased, indicating that the agent learned a more effective path toward the goal.

7. Conclusion

The Q-Learning experiment demonstrates how an agent can learn an optimal policy through trial and error without being explicitly provided with the correct path. After sufficient training, the agent learns to select actions that maximize long-term rewards while avoiding obstacles and unnecessary movements.

Overall Conclusion

The two experiments demonstrate important concepts in Artificial Intelligence and Machine Learning. Decision Tree Classification uses supervised learning to classify data based on learned decision rules, while Q-Learning uses reinforcement learning to learn an optimal policy through interaction with an

environment. Both experiments show how computational algorithms can learn from data or experience and make effective decisions.

References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
2. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
3. Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
4. [NumPy Documentation](#)
5. [Matplotlib Documentation](#)
6. [Scikit-learn Documentation](#)
7. Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8, 279–292.